

软件系统设计

Software Systems Design

design and
ngly
ch

of software
and

Quality assurance
10.30%

Requirements
engineering 4.72%

Software design 1.29%

Software management
0.43%

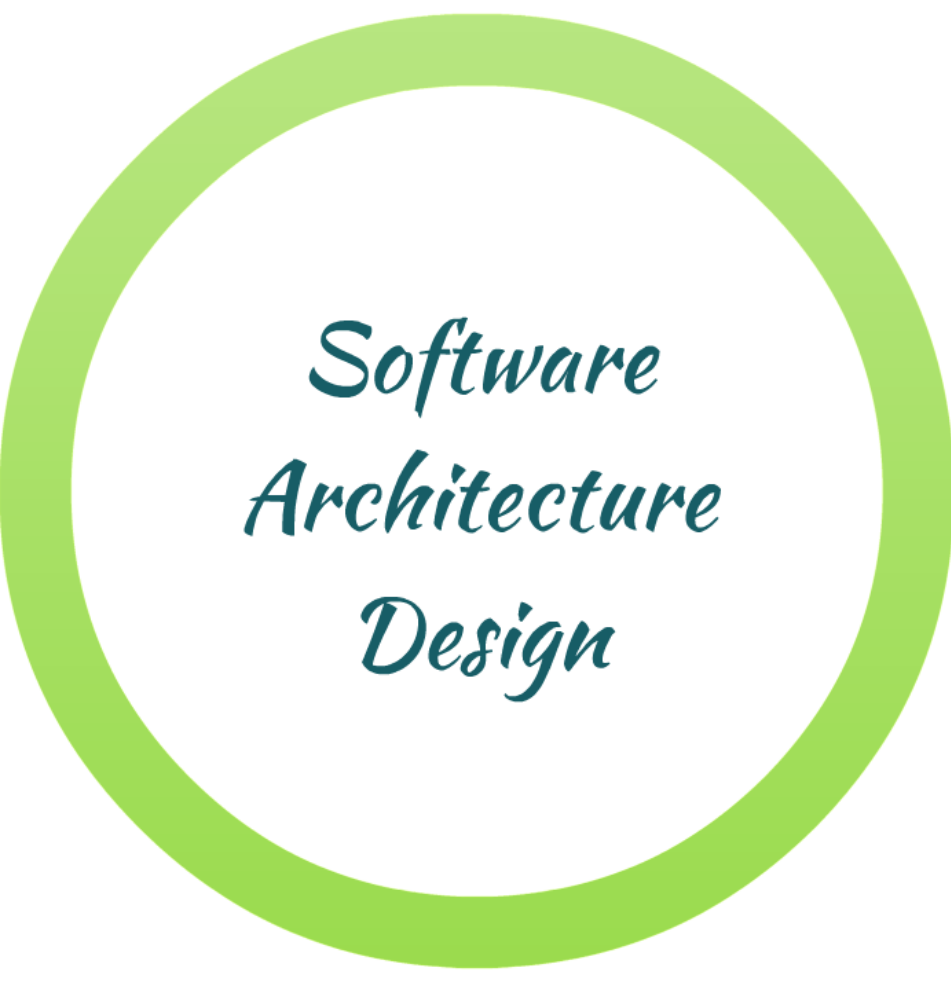
Software Engineering

air

gn and
or through

design
hitecture
ware
and
g topic





*Software
Architecture
Design*



*Software
Design
Patterns*



Course Logistics

Who am I?

Lecturer in Charge:

张贺, He (Jason) Zhang

Office: 费彝民楼B座812室

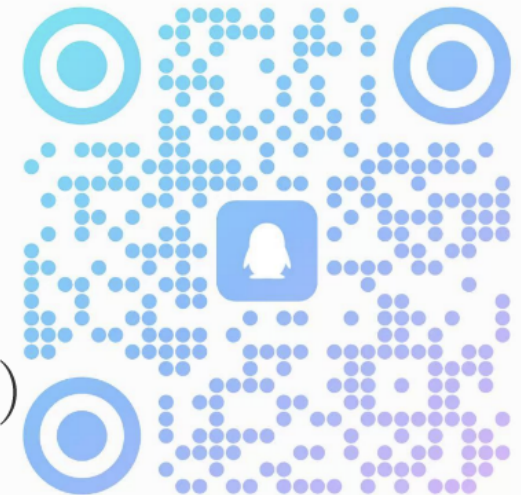
Phone: 83621360/61/62/63 (812)

Email: hezhang@nju.edu.cn

Web: <https://softeng.nju.edu.cn/faculty/HeZhang/>



2025本科系统设计课...



Why Study Software Design & Architecture?

- Software (IT) systems are everywhere
- Every **software intensive system** has a software design and architecture
- Software design and architecture are an increasingly important area of practice, education, and research
- As a profession: **Software Architect**
- As a research area:
 - Originally started around 1960
 - Attracting major attention since 1990
- This course is about
 - Concepts, principles, methods, and patterns of software design and architecture
 - State-of-the-art practices of software design and architecture

TOP1-3 of the Best Jobs in America since 2010s

Best Jobs in America

CNNMoney/PayScale's top 100 careers with big growth, great pay and satisfying work.



1 of 100

1. Software Architect

NEXT

Median pay: \$124,000

Top pay: \$169,000

10-year job growth: 23%

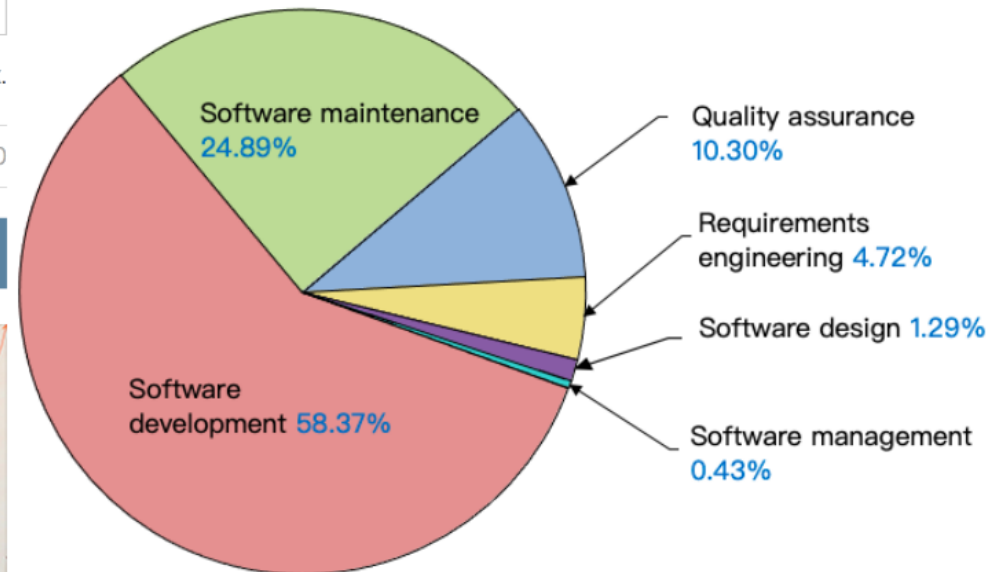
In the same way an architect designs a house, software architects lay out a design plan for new programs. That usually means leading a team of developers and engineers, and making sure all the pieces come together to make fully-functioning software.



Christopher Felpel, software architect/consultant

What's great: New problems come up all the time and new technologies arise, making each day different, and keeping professionals in demand. "I'm pinged at least once or twice a week for new opportunities," said software architect Christopher Felpel. "There's just a lot of work out there, and that doesn't surprise me." --Jillian Eugenios

<http://money.cnn.com/pf/best-jobs/>



Large Language Models for Software Engineering



Learning Objectives

- Understand **concepts** and **principles** of software design and architecture
- Create software architecture by taking requirements or through reverse architecting
- Apply **patterns**, **styles**, **middleware technologies** and **frameworks** in creating software architecture and design
- **Analyze** software design and **evaluate** software architecture systematically
- Understand state-of-the-art **methods** applied in software design and architecture
- Understand **relationships** between software design and software architecture, and other software engineering topic areas

Course Design 1

- Sources
 - SECS (Software Engineering Curriculum Standard)
 - Section 4.11: Software Modelling and Analysis
 - Section 4.12: Software Design
 - Strong relationships with other areas
 - SWEBOK (Software Engineering Body of Knowledge)
 - Chapter 3: Software Design
- Previous Courses
 - UNSW COMP9117
 - USYD INFO3220
 - ITU & SMU
 - NJU 2014-2024
- **2025**
 - 2025 April (Week 8 - Week 11): Software Architecture Concepts, Quality Attributes & Architecture Patterns
 - 2025 May (Week 12 - Week 15): Design Methods, Architecture Evaluation, Microservices Architecture
 - 2025 June (Week 16): Course Review

Course Design 2

- Lectures
 - Interaction is preferred in the class
- Assignments
 - "Apply" the principles and methods you have learned
 - Conduct independent research on "technologies"
 - Leverage your programming and technical background
 - All assignments will be plagiarism-checked!
- Group exercises
 - Students will be assigned to groups for assignments
 - These sessions are meant to reinforce learning about the lecturing materials and help you to gain understanding and skills
 - For group exercises, you have to create and/or evaluate a software architecture with partial design systematically

Final Mark

- Final exam composition:
 - 50% from architecture design
 - 50% from design patterns
- Final mark = **final exam** * 60% + **assignments** * 40%
- Final exam comprising a mixture of
 - Concise questions
 - Knowledge: recall concepts, principles and methods
 - Comprehension: interpret, compare, contrast ...
 - Design and evaluation exercises:
 - Comprehension: translate knowledge to new context, infer causes ...
 - Application: apply what you have learned in new concrete situation ...
 - Necessary reference material will be supplied

References: (no official textbooks)

- L. Bass, P. Clements and R. Kazman (2013/2022) *Software Architecture in Practice*, 3rd/4th Edition, Addison-Wesley
- R.N. Taylor, N. Medvidovic and E.M. Dashofy (2010) *Software Architecture: Foundations, Theory, and Practice*, John Wiley & Sons
- H. Cervantes and R. Kazman (2024) *Designing Software Architecture: A Practical Approach*, 2nd Edition, Addison-Wesley
- A. Shalloway and J. Trott (2004) *Design Patterns Explained: A New Perspective on Object Oriented Design*, 2nd Edition, Addison-Wesley
- P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Merson, R. Nord, and J. Stafford (2011) *Documenting Software Architectures: Views and Beyond*, 2nd Edition, Addison-Wesley
- F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal (1996) *Pattern-Oriented Software Architecture Volume 1: A System of Patterns*, John Wiley & Sons

References: (no official textbooks)

- L. Bass, P. Clements and R. Kazman (2013/2022) *Software Architecture in Practice*, 3rd/4th Edition, Addison-Wesley
- R.N. Taylor, N. Medvidovic and E.M. Dashofy (2010) *Software Architecture: Foundations, Theory, and Practice*, John Wiley & Sons
- H. Cervantes and R. Kazman (2024) *Designing Software Architecture: A Practical Approach*, 2nd Edition, Addison-Wesley
- A. Shawlaway and J. Trott (2004) *Design Patterns Explained: A New Perspective on Object Oriented Design*, 2nd Edition, Addison-Wesley
- P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Merson, R. Nord, and J. Stafford (2011) *Documenting Software Architectures: Views and Beyond*, 2nd Edition, Addison-Wesley
- F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal (1996) *Pattern-Oriented Software Architecture Volume I: A System of Patterns*, John Wiley & Sons

- ## References: (no official textbooks)
- L. Bass, P. Clements and R. Kazman (2013/2022) *Software Architecture in Practice*, 3rd/4th Edition, Addison-Wesley
 - R.N. Taylor, N. Medvidovic and E.M. Dashofy (2010) *Software Architecture: Foundations, Theory, and Practice*, John Wiley & Sons
 - H. Cervantes and R. Kazman (2024) *Designing Software Architecture: A Practical Approach*, 2nd Edition, Addison-Wesley
 - A. Shawlaway and J. Trott (2004) *Design Patterns Explained: A New Perspective on Object Oriented Design*, 2nd Edition, Addison-Wesley
 - P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Merson, R. Nord, and J. Stafford (2011) *Documenting Software Architectures: Views and Beyond*, 2nd Edition, Addison-Wesley
 - F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal (1996) *Pattern-Oriented Software Architecture Volume I: A System of Patterns*, John Wiley & Sons

Who am I?

Lecturer in Charge:

张贺, He (Jason) Zhang

Office: 费彝民楼B座812室
Phone: 83621360/61/62/63 (812)
Email: hezhang@nju.edu.cn
Web: <https://softeng.nju.edu.cn/faculty/HeZhang/>

2025本科生的设计课...

Who am I?

Lecturer in Charge:

张贺, He (Jason) Zhang

Office: 费彝民楼B座812室
Phone: 83621360/61/62/63 (812)
Email: hezhang@nju.edu.cn
Web: <https://softeng.nju.edu.cn/faculty/HeZhang/>



2025本科生的设计课...

Who am I?

Lecturer in Charge:

张贺, He (Jason) Zhang

Office: 费彝民楼B座812室
Phone: 83621360/61/62/63 (812)
Email: hezhang@nju.edu.cn
Web: <https://softeng.nju.edu.cn/faculty/HeZhang/>

2025本科生的设计课...



Who am I?

Lecturer in Charge:

张贺, He (Jason) Zhang

Office: 费彝民楼B座812室
Phone: 83621360/61/62/63 (812)
Email: hezhang@nju.edu.cn
Web: <https://softeng.nju.edu.cn/faculty/HeZhang/>



2025本科生的设计课...

Who am I?

Lecturer in Charge:

张贺, He (Jason) Zhang

Office: 费彝民楼B座812室
Phone: 83621360/61/62/63 (812)
Email: hezhang@nju.edu.cn
Web: <https://softeng.nju.edu.cn/faculty/HeZhang/>

2025本科生的设计课...



Who am I?

Lecturer in Charge:

张贺, He (Jason) Zhang

Office: 费彝民楼B座812室
Phone: 83621360/61/62/63 (812)
Email: hezhang@nju.edu.cn
Web: <https://softeng.nju.edu.cn/faculty/HeZhang/>



2025本科生的设计课...

Who am I?

Lecturer in Charge:

张贺, He (Jason) Zhang

Office: 费彝民楼B座812室
Phone: 83621360/61/62/63 (812)
Email: hezhang@nju.edu.cn
Web: <https://softeng.nju.edu.cn/faculty/HeZhang/>

2025本科生的设计课...



Why Study Software Design & Architecture?

- Software (IT) systems are everywhere
- Every **software intensive system** has a software design and architecture
- Software design and architecture are an increasingly important area of practice, education, and research
- As a profession: **Software Architect**
- As a research area:
 - Originally started around 1960
 - Attracting major attention since 1990
- This course is about
 - Concepts, principles, methods, and patterns of software design and architecture
 - State-of-the-art practices of software design and architecture

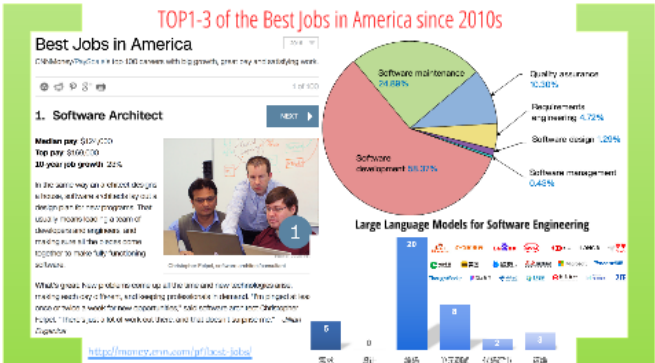
- ## Why Study Software Design & Architecture?
- Software (IT) systems are everywhere
 - Every **software intensive system** has a software design and architecture
 - Software design and architecture are an increasingly important area of practice, education, and research
 - As a profession: **Software Architect**
 - As a research area:
 - Originally started around 1960
 - Attracting major attention since 1990
 - This course is about
 - Concepts, principles, methods, and patterns of software design and architecture
 - State-of-the-art practices of software design and architecture

Final Mark

- Final exam composition:
 - 50% from architecture design
 - 50% from design patterns
- Final mark = **final exam** * 60% + **assignments** * 40%
- Final exam comprising a mixture of
 - Concise questions
 - Knowledge: recall concepts, principles and methods
 - Comprehension: interpret, compare, contrast ...
 - Design and evaluation exercises:
 - Comprehension: translate knowledge to new context, infer causes ...
 - Application: apply what you have learned in new concrete situation ...
- Necessary reference material will be supplied

- ## Final Mark
- Final exam composition:
 - 50% from architecture design
 - 50% from design patterns
 - Final mark = **final exam** * 60% + **assignments** * 40%
 - Final exam comprising a mixture of
 - Concise questions
 - Knowledge: recall concepts, principles and methods
 - Comprehension: interpret, compare, contrast ...
 - Design and evaluation exercises:
 - Comprehension: translate knowledge to new context, infer causes ...
 - Application: apply what you have learned in new concrete situation ...
 - Necessary reference material will be supplied

Course Logistics

[illegible]

Course Design 2

- Lectures
 - Interaction is preferred in the class
- Assignments
 - "Apply" the principles and methods you have learned
 - Conduct independent research on "technologies"
 - Leverage your programming and technical background
 - All assignments will be plagiarism-checked!
- Group exercises
 - Students will be assigned to groups for assignments
 - These sessions are meant to reinforce learning about the lecturing materials and help you to gain understanding and skills
 - For group exercises, you have to create and/or evaluate a software architecture with partial design systematically

- ## Course Design 2
- Lectures
 - Interaction is preferred in the class
 - Assignments
 - "Apply" the principles and methods you have learned
 - Conduct independent research on "technologies"
 - Leverage your programming and technical background
 - All assignments will be plagiarism-checked!
 - Group exercises
 - Students will be assigned to groups for assignments
 - These sessions are meant to reinforce learning about the lecturing materials and help you to gain understanding and skills
 - For group exercises, you have to create and/or evaluate a software architecture with partial design systematically

Course Design 1

- Sources
 - SECS (Software Engineering Curriculum Standard)
 - Section 4.11: Software Modelling and Analysis
 - Section 4.12: Software Design
 - Strong relationships with other areas
 - SWEBOK (Software Engineering Body of Knowledge)
 - Chapter 3: Software Design
- Previous Courses
 - UNSW COMP9117
 - USYD INFO3220
 - ITU & SMU
 - NJU 2014-2024
- 2025
 - 2025 April (Week 8 - Week 11): Software Architecture Concepts, Quality Attributes & Architecture Patterns
 - 2025 May (Week 12 - Week 15): Design Methods, Architecture Evaluation, Microservices Architecture
 - 2025 June (Week 16): Course Review

- # Course Design 1
- Sources
 - SECS (Software Engineering Curriculum Standard)
 - Section 4.11: Software Modelling and Analysis
 - Section 4.12: Software Design
 - Strong relationships with other areas
 - SWEBOK (Software Engineering Body of Knowledge)
 - Chapter 3: Software Design
 - Previous Courses
 - UNSW COMP9117
 - USYD INFO3220
 - ITU & SMU
 - NJU 2014-2024
 - 2025
 - 2025 April (Week 8 - Week 11): Software Architecture Concepts, Quality Attributes & Architecture Patterns
 - 2025 May (Week 12 - Week 15): Design Methods, Architecture Evaluation, Microservices Architecture
 - 2025 June (Week 16): Course Review

Learning Objectives

- Understand **concepts** and **principles** of software design and architecture
- Create software architecture by taking requirements or through reverse architecting
- Apply **patterns**, **styles**, **middleware technologies** and **frameworks** in creating software architecture and design
- **Analyze** software design and **evaluate** software architecture systematically
- Understand state-of-the-art **methods** applied in software design and architecture
- Understand **relationships** between software design and software architecture, and other software engineering topic areas

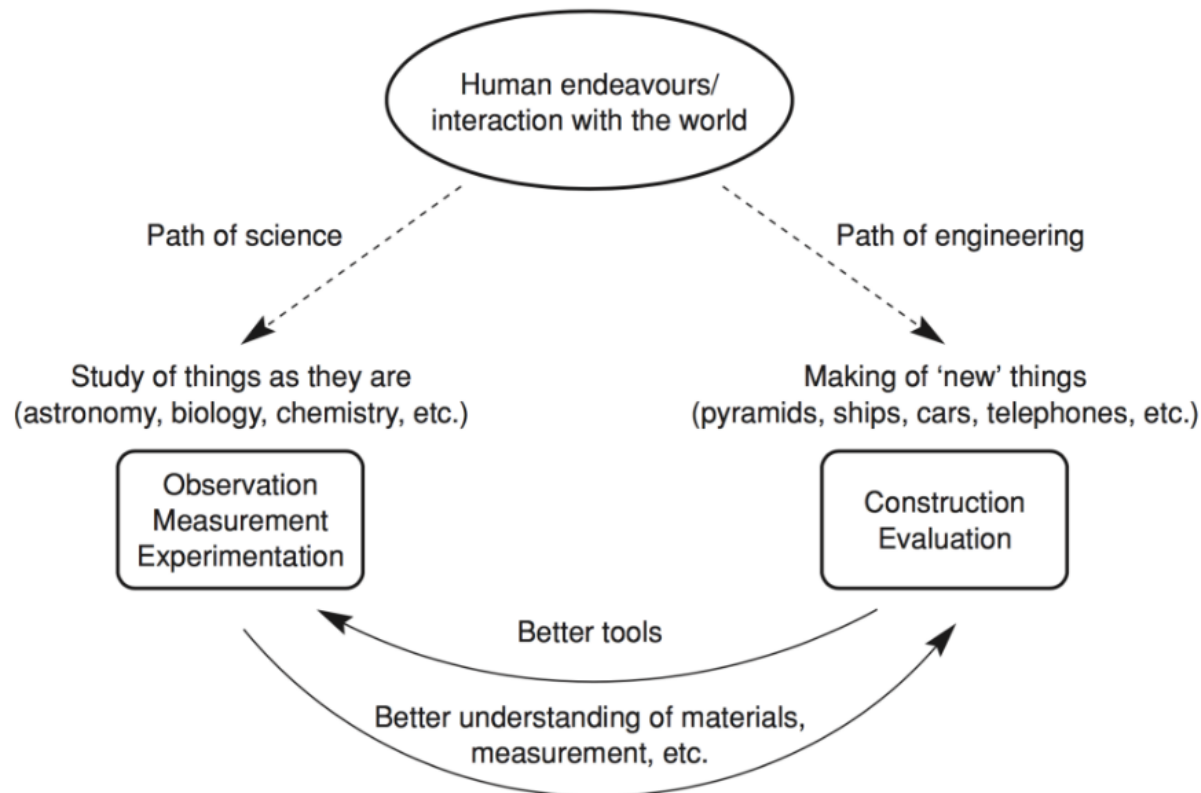
- ## Learning Objectives
- Understand **concepts** and **principles** of software design and architecture
 - Create software architecture by taking requirements or through reverse architecting
 - Apply **patterns**, **styles**, **middleware technologies** and **frameworks** in creating software architecture and design
 - **Analyze** software design and **evaluate** software architecture systematically
 - Understand state-of-the-art **methods** applied in software design and architecture
 - Understand **relationships** between software design and software architecture, and other software engineering topic areas



Introduction

Understanding Software Engineering

- Software | Engineering
 - Software vs. ?
 - Science vs. Engineering

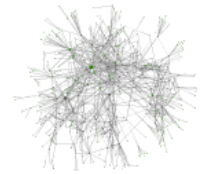


What is Software Architecture?

- Definition 1: “The software architecture of a program or computing system is the structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them.” [Software Engineering Institute (SEI)]
- Definition 2: “The fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution.” [IEEE 1471 -2000 “Recommended Practice for Architectural Description of Software-Intensive Systems”]

Architecture vs. Design

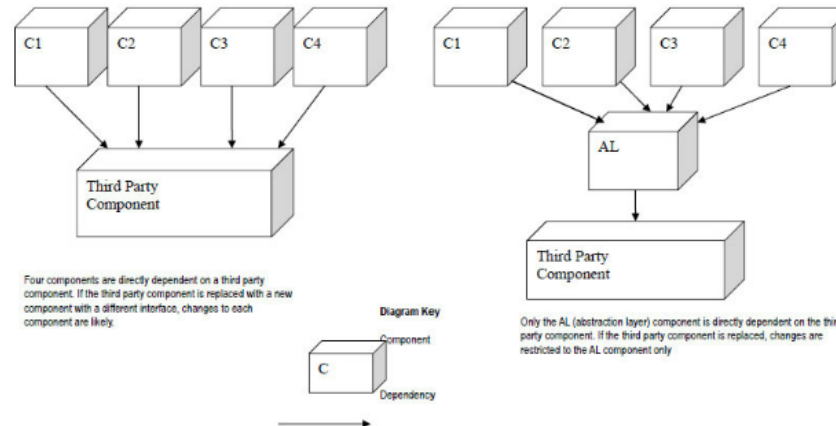
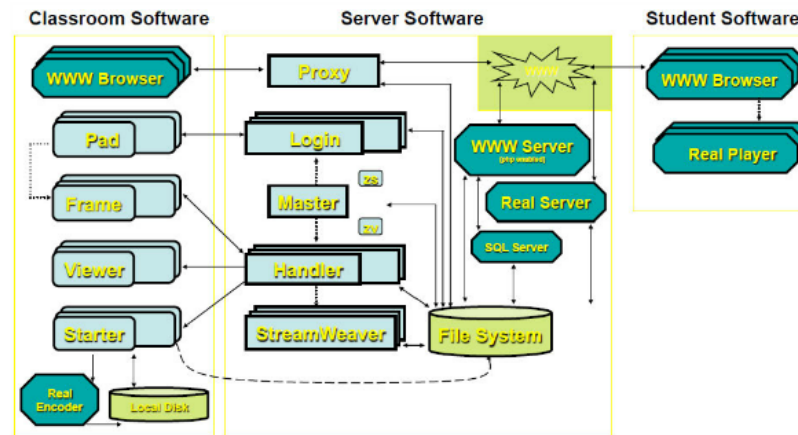
- It's about software design
 - All architecture is software design, but not all design is software architecture
 - "Architecting" is part of the design process
- Other views:
 - **High-level** designs
 - A set of design decisions
- Structure/Organization of the system
 - Elements: components & connectors
 - Relationships: static & dynamic relationships
- Properties: elements, groups of elements & overall system



Architecture vs. Structure

- Decomposition of system into components/modules/subsystems
- Architecture defines:
 - Component **interfaces**
 - What a component can do?
 - Component **communications** and **dependencies**
 - How components communicate?
 - Component **responsibilities**
 - Precisely what a component will do when you ask it?

Structure and Architecture



Architecture Specifies Communication

- Communication involves:
 - **Data** passing mechanisms, for example:
 - Function call
 - Remote method invocation
 - Asynchronous message
 - **Control** flow
 - Flow of messages between components to achieve required functionality
 - Sequential
 - Concurrent/parallel
 - Synchronization

Architecture Addresses NFRs

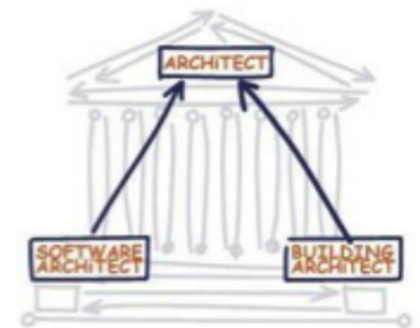
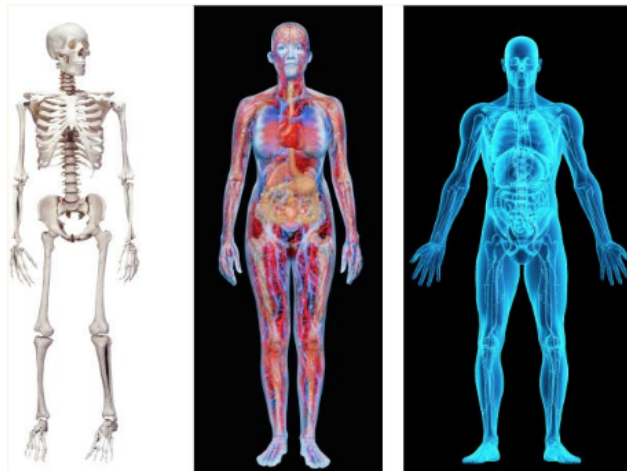
- Non-functional requirements (NFRs) define "how well a system works?"
- NFRs rarely captured in functional requirements
 - Aka. architecture requirements
 - Must be elicited by architect
- NFRs include:
 - Technical constraints
 - Business constraints
 - Quality attributes
- Discussion: A list of quality attributes?

Design is an Abstraction

- Architecture provides an higher level abstract view of a design
 - Hides complexity and implementation of design
 - May or may not be a direct mapping between architecture elements and software elements
- Blackbox design and Whitebox design
 - Informal depiction of system's structure and interactions.
 - Portray the design philosophies embodied in the architecture
- Discussion: Why abstraction in design?

Architecture Views

- A software architecture represents a complex design artifact
- Many possible 'views' of the architecture
 - Analogy with buildings – floor plan, external, electrical, plumbing, air-conditioning



P. Krutchen's 4+1 View Model

- **Logical view**: describes architecturally significant elements of the architecture and the relationships between them.
- **Process view**: describes the concurrency and communications elements of an architecture.
- **Physical view**: depicts how the major processes and components are mapped on to the applications hardware.
- **Development view**: captures the internal organization of the software components as held in e.g., a configuration management tool.
- **Architecture use cases**: capture the requirements for the architecture; related to more than one particular view

How to Develop a Design?

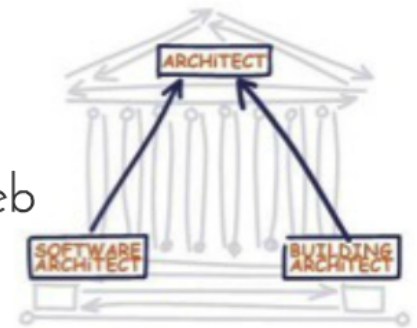
Generic Design Strategies:

- Decomposition
- Abstraction
- Stepwise: 'Divid and Conquer'
- Generate and Test
- Iteration: Incremental Refinement
- Reuseable elements

Architect & Software Architect

“Architects design structures to meet human needs.”- James Fitch, 1972

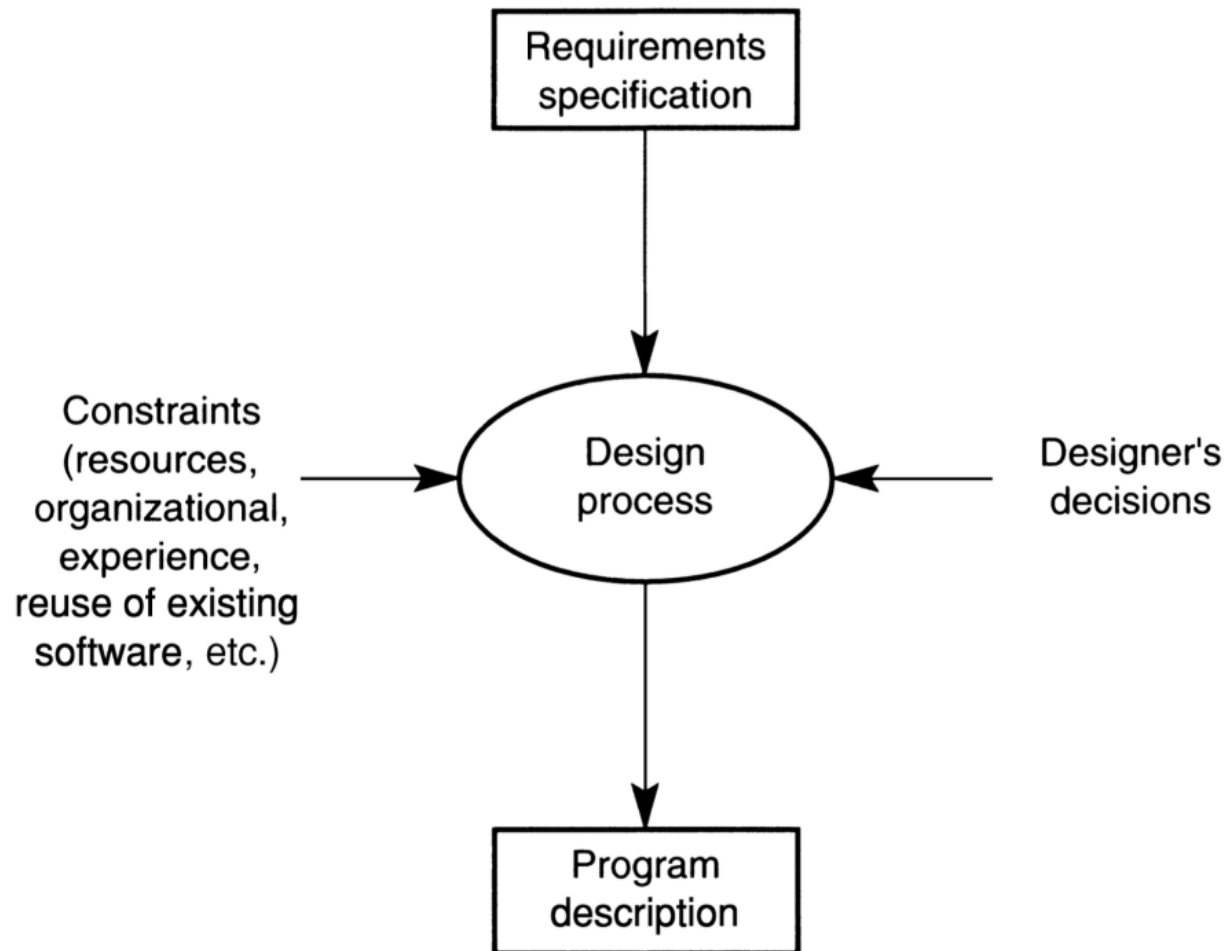
- The role of the architect remains the same
 - Listening to clients, understanding the totality of needs
 - Scrutinizing feasibilities
 - Forming a practical vision of a structure and creating a blueprint
 - Overseeing construction and ensuring compliance to the plan
 - Guiding the vision through the tempest of design changes, crises and ambiguities
- Software architects oversee software construction professionals
 - Programmers, Engineers, Designers
- Famous software architects:
 - Bill Gates: Chief Software Architect of Microsoft
 - Tim Berners-Lee: Inventor and Chief Architect of World Wide Web
 - Roy Fielding: Representational State Transfer (REST)



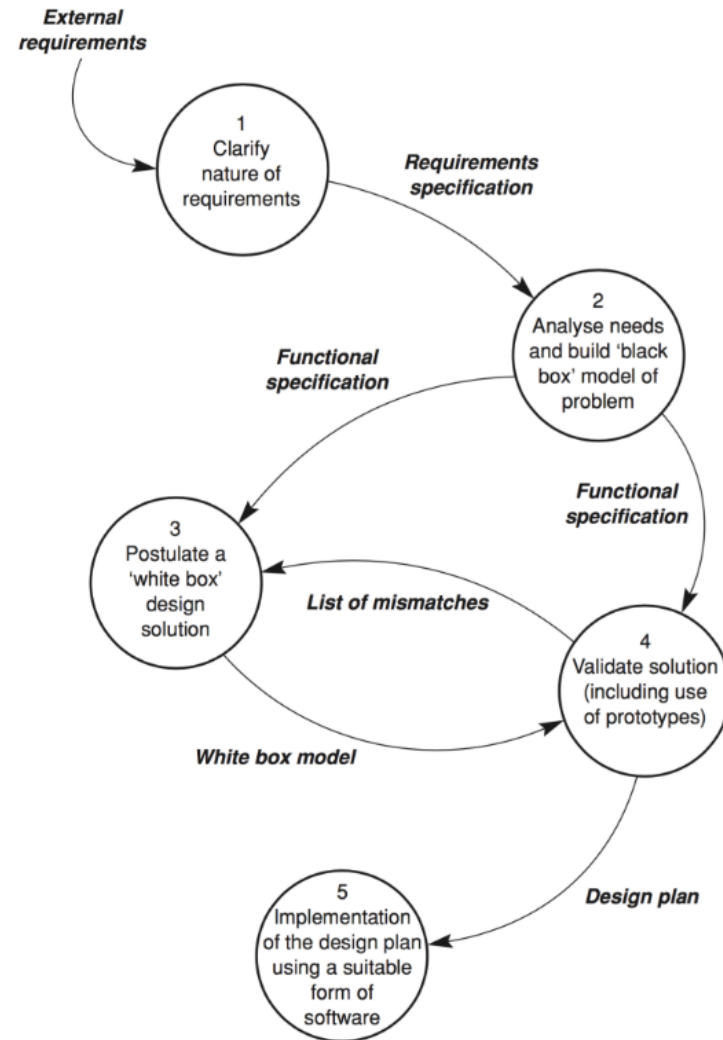
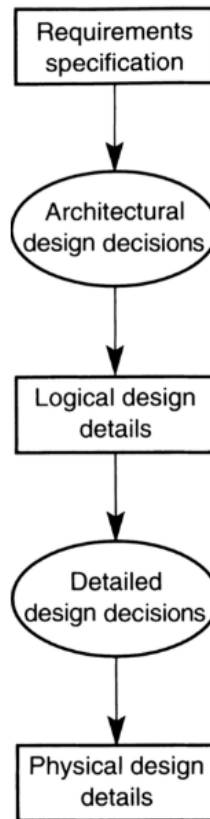
What Does a Software Architect Do?

- **Liaison**
 - Among clients, technical team and business/requirements analysts
 - With management or marketing
- **Software Engineering**
 - Software engineering best practices
- **Technology Knowledge**
 - Deep understanding of technology domain
- **Risk Management**
 - Risks associated with the design, technology choices
 - More?

A General Design Model



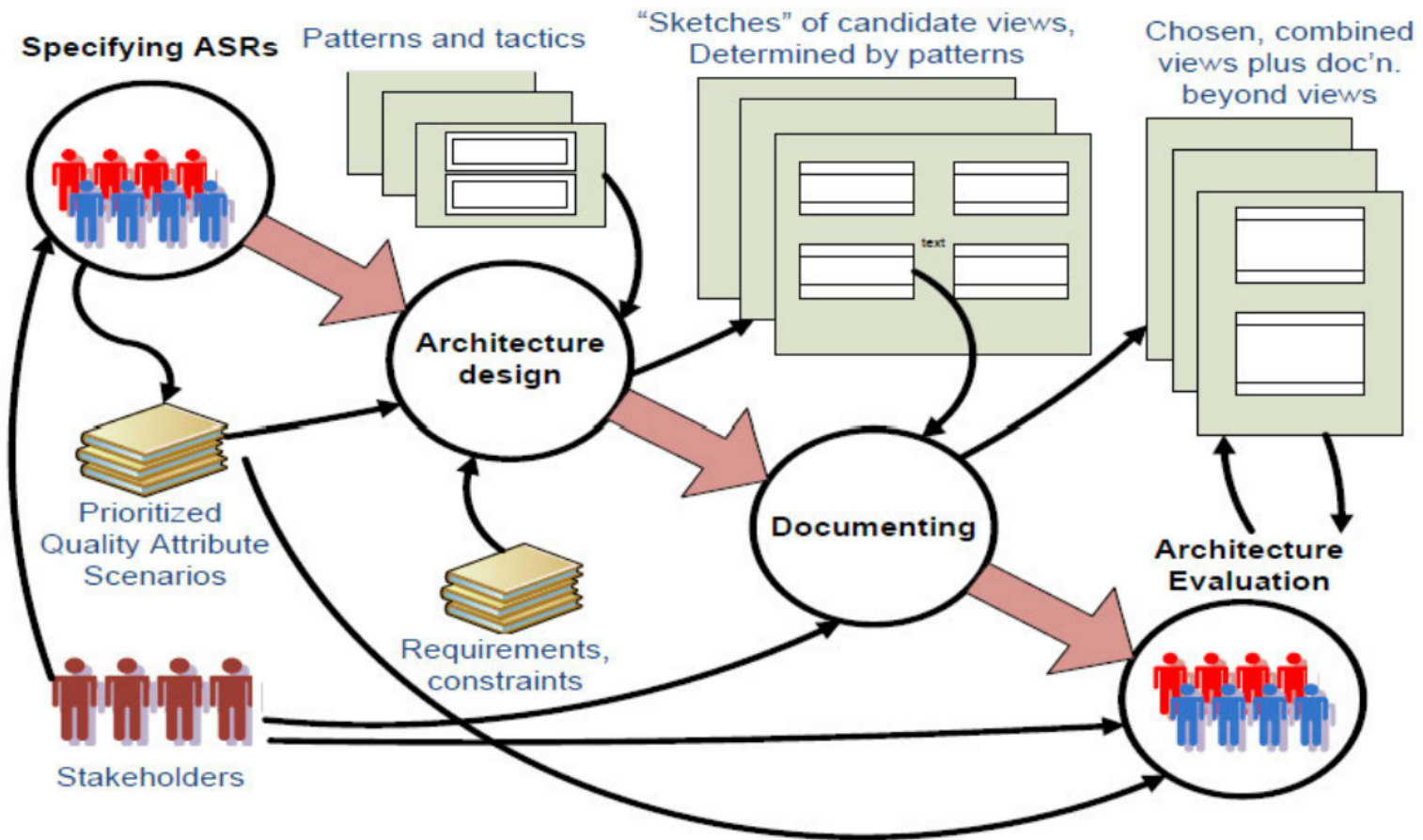
Software Design Process



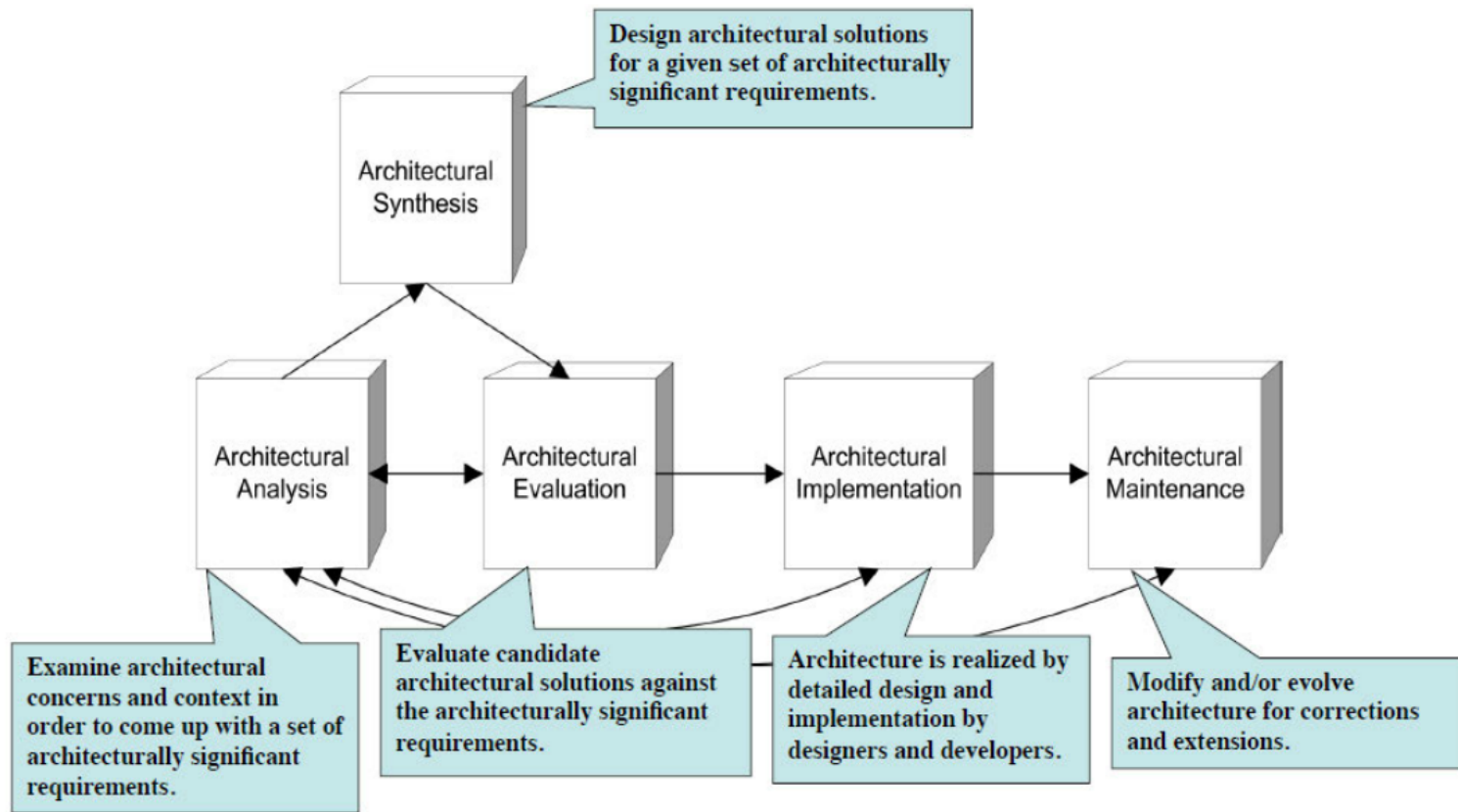
Architecture Activities

- Creating the **business case** for the System
- Understanding the **requirements**
- **Creating** and selecting architecture
- **Communicating** the architecture (stakeholders including developers)
- **Analysing** or **evaluating** the architecture
 - Overall methodologies
 - Quality specific techniques
- **Implementing** the architecture
- Ensuring **conformance** to an architecture

Software Architecture Process



Architecture Lifecycle



Software Design & Architecture Knowledge Areas

- Software Design Basic Concepts
 - General design concepts
 - Context: software development life cycle - requirements, design, construction and testing
 - Design process (role, activity, work product)
 - Enabling techniques for software design
- Key Issues (technical): concurrency, control and handling of events, distribution, exception handling, interactive systems, persistence
- Software Structure and Architecture
 - Architecture Structures and viewpoints
 - Architectural styles and patterns (macro-architecture)
 - Design patterns (micro-architecture)
- Software Design Methods
 - Architecture Methods (e.g., Attribute-Driven Design)
 - Design Methods (e.g., Dynamic System Development Method)

Software Design & Architecture Knowledge Areas

- Software Design Quality Analysis and Evaluation
 - Quality attributes
 - Quality analysis and evaluation methods, techniques and tools
 - Design reviews (e.g. SEI's Architecture Trade-off Analysis Method)
 - Static analysis and dynamic analysis
 - Simulation and prototyping
 - Measures:
 - Metrics: Architecture level
 - Technique specific measures
- Design Modeling and Representation
 - Architecture and Design Notations (Architecture Description Languages (ADL))
 - Unified Modelling Language (UML)
 - Design Documentation (Views & Beyond)
 - Others: differ in ability, focus and domain (e.g. ACME, Rapide)

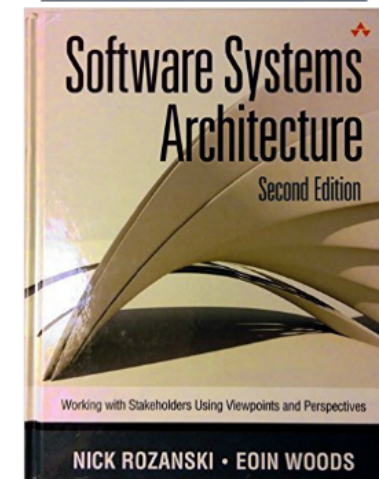
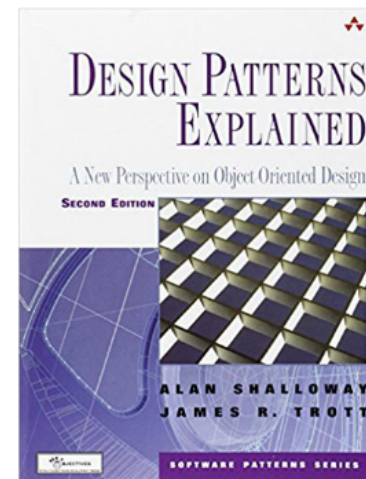
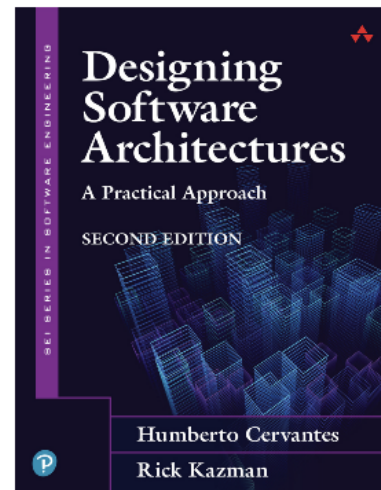
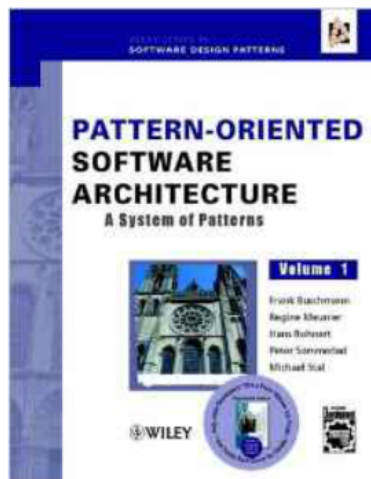
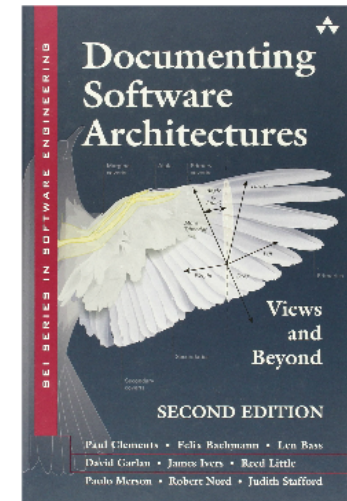
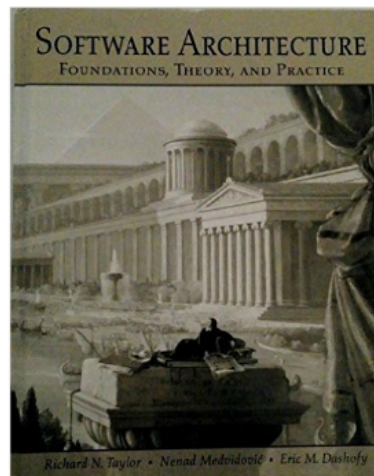
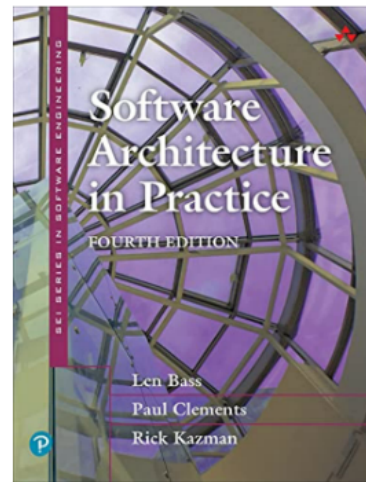
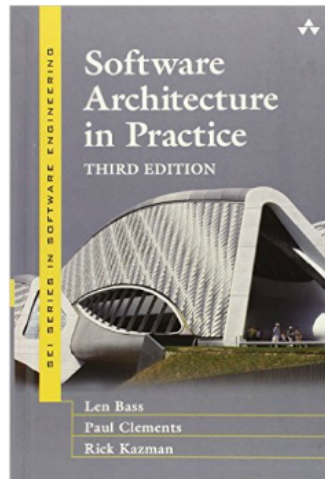
Lectures Preview

- Lecture 1: Introduction
 - Concepts of software architecture
- Lecture 2/3: Quality Attributes and Architecture Tactics
 - Quality attributes, NFRs and scenarios
 - Tactics achieving quality attributes
 - Architecturally significant requirements
- Lecture 4: Architecture Patterns
 - Patterns in software design
 - Architecture styles and patterns
- Lecture 5/6: Architecture Design Methods
 - Design strategies in general
 - Software design process and architecting process
 - ADD method: Attribute Driven Design

Lectures Preview

- Lecture 7: Microservices Architecture
- Lecture 8*: Documenting Software Architecture
 - Modelling notations, views and more
 - Documenting rationales behind your architecture
- Lecture 9*: Evaluating Software Architecture
 - Single attribute specific techniques
 - Trade-off analysis among multiple quality attributes
- Lecture 10: Course Review — Software Architecture
- Final Exam

Reference Books



软件开发效能 实验室



扫码下载
实验室2024年报



扫码下载实验室发布
2024年度调查报告



扫码观看实验室
NJSD'23效能专场



扫码观看实验室
NJSD'24产研专场

欢迎对软件架构研究感兴趣和想成为架构师的同学与我联系

Software Architecture

- **Software Architecture** is the structure or **structures** of the system, which comprise software **elements**, the externally visible **properties** of these components, and the **relationship** among them.
- Box-and-line drawings alone are not architecture, but a starting point.
- Architecture includes behaviour of components.

Role of Architecture

- An architecture is one of the **first artefacts** that represents decision on **how requirements are to be achieved**. As the manifestation of early design decisions, the architecture represents those design decisions that are **hardest to change** and hence deserve the **most careful consideration**.
- An architecture is the key artefact in achieving successful product line engineering, the disciplined development of a family of similar system with less effort, expense, and risk than developing each system independently.
- An architecture is usually the first design artefact to be examined when someone starts working on a system.
- Software architecture provides a **framework** of reference for **maintenance** and **modification** decisions.

Why is software architecture important?

- Software architecture provides a **vehicle for communication**
 - It is a frame of reference in which competing interests may be identified and negotiated
 - **Negotiating requirements** with users
 - **Keeping** customer **informed** of progress, cost etc.
 - **Implementing** management **decisions** and allocations.
- Software architecture manifests the **earliest set of design decisions**
 - It constraints the implementation and developers
 - Implementation must **conform to architecture**
 - **Resource allocation** decisions constrain implementations of individual components

Why is software architecture important?

- Manifestation of early design decisions
 - Software architecture dictates **organisational structure** for development & maintenance efforts, e.g.,
 - Division into **teams**
 - Units for **budgeting**, planning
 - Basis of **W**ork **B**reakdown **S**tructure
 - Organisation for **documentation**
 - Organisation for **CM** libraries
 - Basis of **integration**
 - Basis of **test** plans, testing
 - Basis of **maintenance**

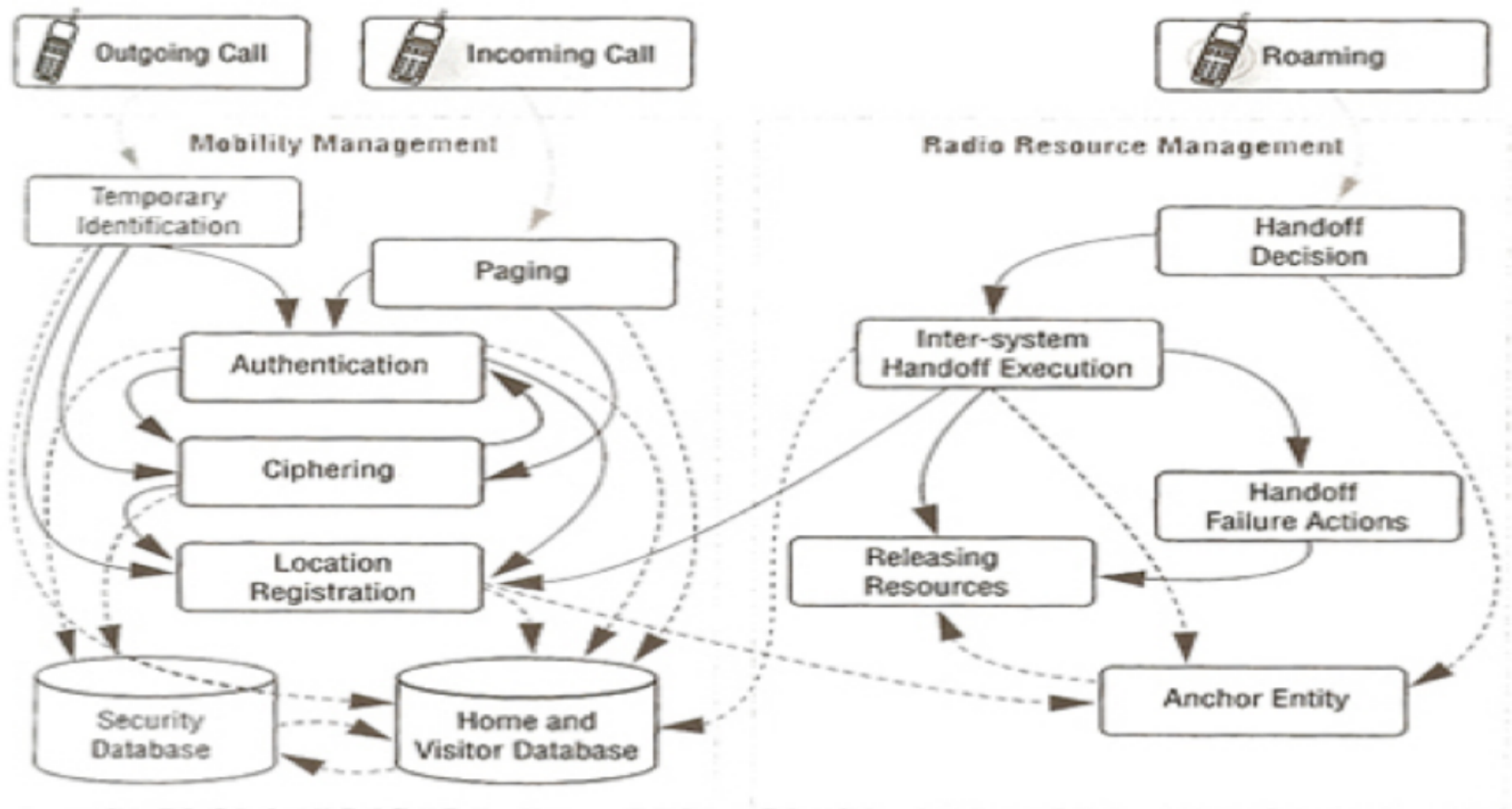
Why is software architecture important?

- Architecture facilitates/hinders **achievement of quality attributes**, e.g., modifiability, security, usability etc.
- Architecture influences qualities, but may **not guarantee them** as there are a number of other factors involved.
- An architecture invokes discussion about **potential change** (80% of effort for a system is **post-deployment effort**)
- Architecture categorise changes into three types:
 - **Local**: signal component modification
 - **Non-local**: several component modification.
 - **Architectural**: modification of the system's basic structure, communication, and coordination mechanism

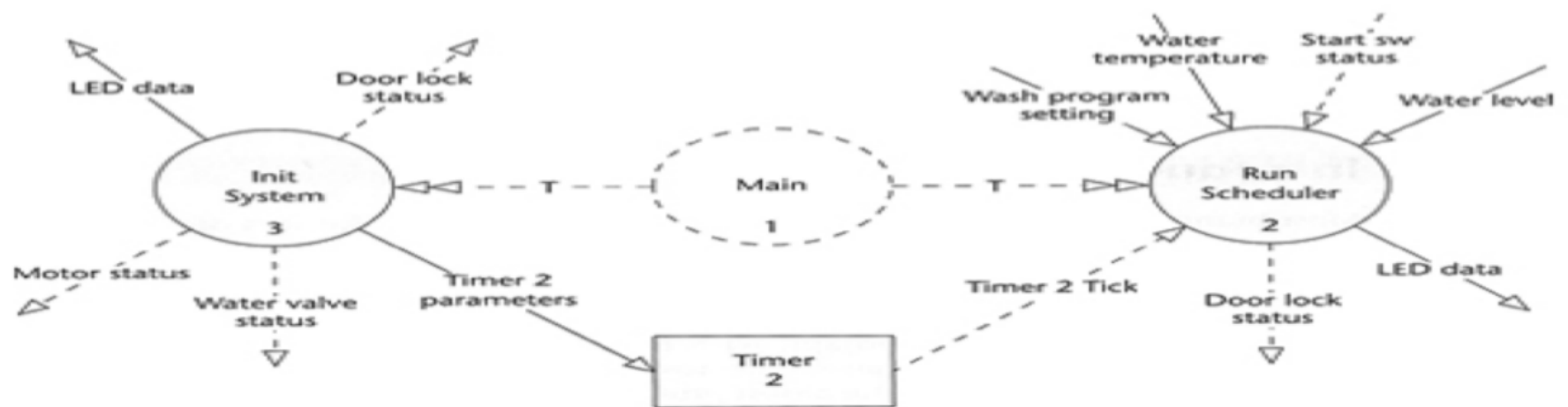
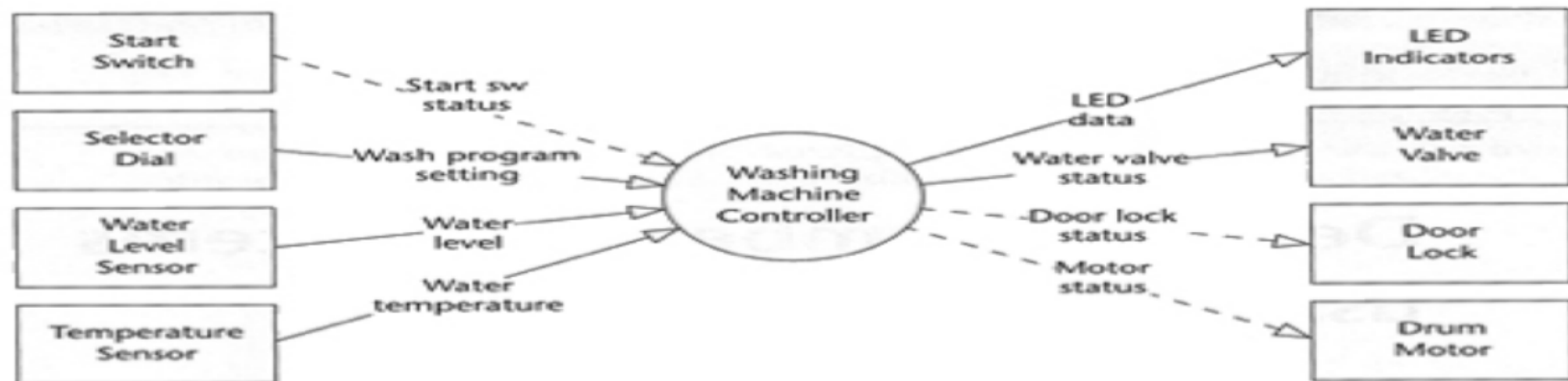
Why is software architecture important?

- Architecture is a **transferable** and **reusable** abstraction – **one-to-many mapping** (one architecture, many systems)
- Architecture is the basis for **product commonality**. A whole product line shares a single architecture
- Systems can be developed by integrating independently developed components via architecture (Component-Based Software Engineering – CBSE)

Mobile Phone System Architecture



Washing Machine Architecture



AI Guardrail Architecture

